

Node.js Complete Notes (বাংলা | Basic → Advanced → Production)

এই নোটটা একদম বাংলায়, step by step বোঝানোর জন্য লেখা। প্রতিটা জিনিসে থাকবে 📌 - কেন লাগে - কোথায় ব্যবহার হয় - কিভাবে ব্যবহার করবে - Real example

লক্ষ্য: তুমি যেন সত্যি সত্যি Node.js বুঝে ফেলো, মুখস্থ না।

1 Node.js আসলে কী?

Node.js হলো **JavaScript runtime environment**। মানে 🖱 Browser ছাড়াই JavaScript চালানো যায়।

কেন Node.js দরকার?

- আগে JS শুধু browser এ চলতো
- Backend লিখতে Java, PHP লাগতো
- Node.js এ একই JS দিয়ে **frontend + backend**

কোথায় ব্যবহার হয়?

- API server
 - Real-time app (chat, notification)
 - File upload/download
 - Microservice
-

2 Node.js Architecture (Event Loop) ✓

Node.js **single-threaded**, কিন্তু slow না। কারণ Event Loop।

Event Loop কী করে?

- একসাথে হাজার request handle করে
- Heavy কাজ অন্য জায়গায় পাঠায়

Flow (সহজ ভাষায়)

1. Call Stack → JS code
2. Node APIs → async কাজ
3. Callback Queue
4. Event Loop → আবার stack এ পাঠায়

কেন এটা জানা জরুরি?

👉 Interview question 👉 Performance issue বুঝতে

3 Blocking vs Non-Blocking

Blocking মানে কী?

একটা কাজ শেষ না হলে পরের কাজ আটকে থাকে 👉

```
const fs = require('fs');
const data = fs.readFileSync('a.txt');
console.log(data.toString());
```

Non-Blocking মানে কী?

একটা কাজ চলাকালীন অন্য কাজ চলতে পারে 👉

```
fs.readFile('a.txt', (err, data) => {
  console.log(data.toString());
});
```

কোথায় কোনটা ব্যবহার করবো?

- 👉 Server এ sync না
 - 👉 Startup script এ sync চলতে পারে
-

4 Module System

কেন Module দরকার?

- Code ছোট ছোট ভাগে ভাগ করতে
- Reuse করতে

CommonJS (পুরনো)

```
module.exports = add;
const add = require('./add');
```

ES Module (Modern)

```
export default add;  
import add from './add.js';
```

👉 New project এ ES module use করো

5 Core Modules (খুব Important)

fs (File System)

```
fs.writeFile('a.txt', 'hello', () => {});
```

👉 File read/write

path

```
path.join(__dirname, 'file.txt');
```

👉 OS independent path

http

👉 Raw server বানাতে

events

👉 Custom event system

6 HTTP Server (Raw Node)

```
const http = require('http');  
http.createServer((req, res) => {  
  res.end('Hello');  
}).listen(3000);
```

কেন এটা শেখা দরকার?

👉 Express ভিতরে কী করে বুঝতে

7 Async Programming (Heart of Node)

Problem (Callback Hell)

Solution → Promise → Async/Await

```
async function getUser() {  
  const user = await fetchUser();  
  return user;  
}
```

👉 Clean + readable code

8 Error Handling (Production Way)

```
process.on('uncaughtException', err => {  
  console.log('Error:', err);  
});
```

কেন দরকার?

👉 Server crash আটকাতে

9 Environment Variable

```
process.env.PORT
```

কেন দরকার?

- Secret hide
 - Config change without code
-

10 Stream (Performance King)

```
fs.createReadStream('big.mp4').pipe(res);
```

কেন Stream?

- বড় file
 - কম memory
-

1 1 Cluster (Scalability)

👉 CPU এর সব core use করতে

1 2 Security Best Practice

- Input validation
 - Env secret
 - Rate limit
-

1 3 Production Folder Structure

```
src/  
├─ controllers/  
├─ services/  
├─ utils/  
└─ app.js
```



শেষ কথা

এই Node.js নোট শেষ করলে তুমি: - Node core বুঝবে - Express সহজ লাগবে - Backend confident হবে

👉 Next তুমি চাইলে: - Express.js (একইভাবে বাংলায়) - Node + Express + Mongoose Full Project - Auth System Deep